

实验三：用户态线程（协程）（学生版）

1. 实验目标

- 打破黑盒**：通过手动操作理解 CPU 上下文（指令寄存器 PC、栈指针 SP、通用寄存器等）。
- 理解调用栈**：掌握为每个执行流分配独立栈空间的原理，理解局部变量的存储机制。
- 连接工业技术**：理解 Go 语言 Goroutine 及 Python `async/await` 的底层核心原理。
- 掌握调度算法**：实现基础的轮转调度（Round-Robin），并探索抢占式调度。

2. 实验环境准备

- 操作系统**：Ubuntu 22.04 LTS 或更高版本。
- 编译器**：`gcc`。
- 核心头文件**：`<ucontext.h>`（POSIX 标准库，请确保系统已安装 `libc6-dev`）。
- libc6-dev 的安装**：`sudo apt update`, `sudo apt install libc6-dev`

3. 实验任务

任务一：初识 `ucontext`

在开始编写协程库前，请先阅读 `Exp3_参考手册`，理解 `ucontext_t` 结构体及 `swapcontext` 等函数的作用。

任务二：补全极简协程库

本任务要求你在 `coroutine.c` 框架代码中补全 13 处关键逻辑。

框架代码说明

协程库的核心组件包括：

- `Coroutine` 结构体：即协程控制块（TCB），保存上下文、栈空间及运行状态。
- `Scheduler` 结构体：管理所有协程的槽位及当前正在运行的 ID。
- `scheduler_init`：初始化调度器，把所有协程槽位标记为 `FREE`，表示这些位置还没有被使用。
- `coroutine_wrapper`：协程的入口包装函数，负责在任务结束时清理状态并切回调度器。
- `coroutine_spawn`：创建一个新协程，完成槽位分配、上下文初始化、独立栈设置和入口函数绑定。
- `coroutine_resume`：把某个处于 `RUNNABLE` 或 `SUSPEND` 状态的协程切到 CPU 上继续执行。
- `coroutine_yield`：让当前正在运行的协程主动保存现场并把控制权交还给主调度器。

待补全代码 `coroutine.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <ucontext.h>

#define MAX_COROUTINES 10
#define STACK_SIZE (1024 * 64)

typedef enum { FREE, RUNNABLE, RUNNING, SUSPEND } State;

typedef struct {
    ucontext_t ctx;          // 保存该协程的寄存器现场、栈信息和返回去向
    char stack[STACK_SIZE]; // 该协程独享的栈空间
    State state;             // 协程当前所处状态
    void (*func)(void);      // 协程真正要执行的任务函数
} Coroutine;

struct Scheduler {
    ucontext_t main_ctx;          // 主调度器上下文, yield/结束后都会切回这里
    Coroutine coroutines[MAX_COROUTINES]; // 协程槽位表
    int current_id;               // 当前正在运行的协程 ID, -1 表示当前在 main_ctx 中
} scheduler;

void scheduler_init() {
    scheduler.current_id = -1;
    // ★ TODO 1: 将所有协程状态初始化为 FREE
}

void coroutine_wrapper() {
    int id = scheduler.current_id;

    // 当前协程第一次被 resume 时, 会从 wrapper 开始执行。
    // wrapper 不是业务函数, 它的第一步是根据 id 找到当前协程, 并调用真正的任务函数。
    // ★ TODO 2: 调用 scheduler.coroutines[id] 绑定的 func()

    // 如果 func() 返回, 说明这个协程的任务已经执行完毕。
    // 这时必须释放槽位, 否则调度器会一直认为该协程还没结束。
    // ☆ TODO 3: 将当前协程状态置为 FREE, 并将 current_id 置为 -1

    // wrapper 不能直接 return 到一个不明确的位置。
    // 这里显式恢复主调度器上下文, 让 main 中的调度循环继续扫描其他协程。
    // ★ TODO 4: 使用 setcontext(&scheduler.main_ctx) 切回主调度器
}

int coroutine_spawn(void (*func)(void)) {
    int id = -1;
    // ★ TODO 5: 遍历槽位, 找到一个状态为 FREE 的协程 ID
    // 如果找不到, 直接返回 -1

    Coroutine *co = &scheduler.coroutines[id];
    co->state = RUNNABLE;
    co->func = func;
```

```

// getcontext 先把“当前 CPU 上下文”保存到 co->ctx，后面再把它改造成一个新协程入口。
getcontext(&co->ctx);
// ★ TODO 6: 设置协程独立栈: uc_stack.ss_sp 指向 co->stack, uc_stack.ss_size 为相应大小
// 说明: 协程切进来后, 局部变量和函数调用都会压在这块栈上。

// ☆ TODO 7: 设置 uc_link。当协程退出（非 yield）后, 硬件会自动切往此地址。
// 建议设置为 &scheduler.main_ctx 或 NULL（此处我们将通过 wrapper 手动切回）
// 说明: uc_link 可以理解为“这个上下文自然结束后, 下一个该回到哪里”。

// ★ TODO 8: 使用 makecontext 让协程从 coroutine_wrapper 开始执行
// 说明: makecontext 会修改 co->ctx 的入口地址, 使其第一次被 resume 时从 wrapper 开始跑。
return id;
}

void coroutine_resume(int id) {
    if (id < 0 || id >= MAX_COROUTINES) return;
    Coroutine *co = &scheduler.coroutines[id];
    // ★ TODO 9: 如果状态不是 RUNNABLE 或 SUSPEND, 直接 return

    co->state = RUNNING;
    scheduler.current_id = id;
    // ★ TODO 10: 使用 swapcontext 保存当前(main)上下文并切换到协程上下文
    // 说明: swapcontext(&main_ctx, &co->ctx) 会先保存当前主调度器现场, 再跳到协程继续执行。
}

void coroutine_yield() {
    int id = scheduler.current_id;
    // ★ TODO 11: 如果当前处于调度器环境 (id == -1), 直接 return

    Coroutine *co = &scheduler.coroutines[id];
    // ☆ TODO 12: 将当前协程状态改为 SUSPEND, current_id 重置为 -1

    // ★ TODO 13: 使用 swapcontext 切换回调度器上下文 (main_ctx)
    // 说明: 这里是反方向切换, 先保存当前协程现场, 再恢复 main_ctx; 下次 resume 时会从这里之后继续跑。
}

```

任务三：运行与验证

编写测试逻辑，模拟一个需要运行很久的任务。它每处理一批数据就主动 `yield`，让其他协程也有机会执行，避免单个长任务长期占用 CPU。

```

void long_task() {
    int total = 0;

    for (int batch = 1; batch <= 5; batch++) {
        // 模拟一个较长任务的一小批计算。
        // 真实场景中, 这里可能是处理一批请求、一批日志或一段大数组。
        for (int i = 0; i < 1000000; i++) {
            total += i % 7;
        }
    }
}

```

```

    printf("[Long Task] finished batch %d, total = %d\n", batch, total);

    // 每处理完一批就主动让出 CPU。
    // 这样其他协程可以穿插执行，不会一直等到 long_task 完全结束。
    coroutine_yield();
}

printf("[Long Task] all batches done\n");
}

void monitor_task() {
    for (int round = 1; round <= 5; round++) {
        printf("[Monitor] scheduler is still responsive, round %d\n", round);

        // monitor_task 也主动让出 CPU，让 long_task 继续处理下一批。
        coroutine_yield();
    }
}

int main() {
    // 初始化调度器：current_id 置为 -1，所有协程槽位置为 FREE。
    scheduler_init();

    // 创建两个协程。spawn 只是初始化 ctx、独立栈和入口函数，不会立刻执行任务函数。
    coroutine_spawn(long_task);
    coroutine_spawn(monitor_task);

    // 极简轮转调度器：不断扫描协程表，发现未结束的协程就 resume。
    while (1) {
        int all_done = 1;

        for (int i = 0; i < MAX_COROUTINES; i++) {
            if (scheduler.coroutines[i].state != FREE) {
                all_done = 0;

                // resume 会保存 main_ctx，并切换到第 i 个协程的 ctx。
                // 如果该协程之前 yield 过，就会从 yield 返回后继续执行。
                coroutine_resume(i);
            }
        }

        // 扫描一整轮后，如果所有槽位都是 FREE，说明全部协程都执行结束。
        if (all_done) break;
    }

    printf("All coroutines finished.\n");
    return 0;
}

```

编译命令： gcc coroutine.c -o coroutine

预期输出： Long Task 每完成一批后， Monitor 都能穿插输出，说明长任务没有一直独占执行权。

```
[Monitor] scheduler is still responsive, round 3
[Long Task] finished batch 4, total = 11999988
[Monitor] scheduler is still responsive, round 4
[Long Task] finished batch 5, total = 14999985
[Monitor] scheduler is still responsive, round 5
[Long Task] all batches done
All coroutines finished.
```

相比直接使用多个线程，这种协程式分片有几个优势：

- **切换开销更低**：协程切换主要在用户态保存和恢复上下文，不需要每次都进入内核线程调度。
- **并发结构更可控**：只有在明确写出 `coroutine_yield()` 的地方才会切换，临界区和共享变量更容易分析。
- **栈和资源开销更小**：协程栈通常可以比线程栈更小，一个线程内可以管理更多轻量任务。
- **适合 I/O 等待或分批处理场景**：当任务可以拆成多个小阶段时，协程能用顺序代码写出交替推进的效果。

需要注意的是，当前实验实现的是协作式协程。如果某个协程长时间不调用 `yield()`，其他协程仍然会被饿住；这也是后面抢占式调度扩展要解决的问题。

4. 选做挑战（任选其一）

挑战 A：抢占式调度（Preemptive Scheduling）

目前协程只有主动调用 `yield()` 才会让出 CPU。如果某个协程进入死循环，整个程序都会卡死。

- **要求**：利用 Linux 的 `setitimer` 产生时钟中断信号（`SIGALRM`），在信号处理函数中强行调用 `swapcontext`。

挑战 B：具备“时间感知”的调度器

目前调度器在 `while` 循环中不断轮询所有协程。

- **要求**：实现一个 `coroutine_sleep(int seconds)` 函数。调用该函数的协程在指定时间内不会被 `resume`。

5. 实验思考

1. **为什么需要 wrapper 函数？**
2. **栈是怎么增长的？** 在补全代码时，你将栈地址给了 `ucontext_t`，CPU 在切换后如何知道从这块内存的哪个位置开始“压栈”？
3. **局部变量去哪了？** 如果在协程内声明一个 `char arr[1024]`，它是分配在物理内存的哪个区域？会被其他协程污染吗？

6. 实验提交

完成实验报告的填写，以“**学号-姓名-操作系统实验三**”命名

将实验报告提交到作业网站: <https://acm.scu.edu.cn/teach/>